

Ai.Security - AEC-AB

Preview rendered 2026-09-26 02:59 UTC from 11 pages. Generated by the AEC repo agents; the pull request is the source of truth.

Contents

[Security review summary](#) Summary.md

[Findings register](#) Findings.md

[SEC-001: StreamBIM credential storage and FTP transfer security](#) Findings/SEC-001-streambim-credential-storage.md

[SEC-002: Dalux API key handling - broken credential lookup in upload and minor HttpClient gaps](#) Findings/SEC-002-dalux-api-key-handling.md

[SEC-003: Git history audit - no secrets found](#) Findings/SEC-003-git-history-audit-clean.md

[SEC-004: LISPRunner arbitrary AutoCAD Lisp script execution](#) Findings/SEC-004-lisprunner-arbitrary-execution.md

[SEC-005: RunCommand arbitrary AutoCAD command injection](#) Findings/SEC-005-rancmd-arbitrary-execution.md

[SEC-006: DaluxCloudDownload path traversal via server-supplied relative path](#) Findings/SEC-006-dalux-path-traversal.md

[Dependencies](#) Dependencies.md

[Software bill of materials](#) SBOM.md

[Software bill of materials](#) SBOM/README.md

Security review summary

Incomplete pass. Run 20260926-015629 did not finish: the model server stopped answering mid-run. The findings below are what it got to; the rest of the codebase is not reviewed yet. The next nightly sweep picks this repo up again.

Incomplete pass. Run 20260925-214417 did not finish: the model server stopped answering mid-run. The findings below are what it got to; the rest of the codebase is not reviewed yet. The next nightly sweep picks this repo up again.

Rewritten by the AEC security review agent on every run. Internal only.

Scope

Run 20260925-214417 — first pass. Reviewed the StreamBIM credential storage and FTP transfer security area, the Dalux API key handling area, a full git history audit for secrets across all branches, and the Dalux API authentication area.

Run 20260926-015629 — incremental. Reviewed the AutoCAD LISPRunner and RunCommand extensions (Critical RCEs), the DaluxCloudDownload path handling (Medium path traversal), and CI/CD workflows and scripts (verified input validation, path traversal protections, private content scanning).

Counts

Severity	Open	Total
Critical	2	2
High	1	1
Medium	1	1
Low	1	1
Info	1	1
Total	6	6

Status	Count
Open	6

Top open findings

- SEC-004** (Critical) — LISPRunner arbitrary Lisp execution: both script-file and inline modes pass user-supplied content to AutoCAD's `SendStringToExecute()` with zero sanitization. An attacker who can supply the script path or content gains arbitrary AutoCAD Lisp API access.
- SEC-005** (Critical) — RunCommand arbitrary command execution: user-supplied command strings passed directly to AutoCAD's `SendCommand()` with no whitelist, validation, or sanitization. Multi-line input allows command chaining and full AutoCAD API access.
- SEC-002** (High) — Dalux API key handling: broken credential lookup in `DaluxCloudUploadCommand.cs` line 30 (passes `args.ApiKey` with default value "Dalux API Key" instead of the looked-up credential variable to `DaluxApiService`); `HttpClient` gaps (no timeout, no certificate validation callback) in both upload and download services.
- SEC-006** (Medium) — DaluxCloudDownload path traversal via server-supplied relative path: `file.RelativePath` and `file.FileName` from the Dalux API are used directly in `Path.Combine()` without traversal validation. A compromised Dalux API could inject `../` sequences.
- SEC-001** (Low) — StreamBIM credential storage: credentials properly stored in Windows Credential Manager (DPAPI), FTP uses TLS 1.2 Explicit. Minor defense-in-depth gaps: no explicit certificate validation callback on FTP client, password held as plaintext string in memory.

What changed since previous run

Run 20260926-015629 is an incremental pass. Three new Critical and Medium findings were filed (SEC-004, SEC-005, SEC-006). CI/CD workflows and scripts were verified as well-implemented. The backlog has been expanded with additional items identified during review.

Areas not yet covered

- StreamBIM file path validation and injection (StreamBimPathHelper.cs, FailedFile.cs)
- StreamBIM autofill collectors (path validation before use)
- NuGet.config security (insecure source URLs or credentials)
- Directory.Build.props/targets (sensitive defaults)
- Build system command execution (unsanitized args in Build.cs)
- PrintPDF telemetry and logging (PII or secret leakage)
- .NET package versions (SBOM review)
- API version skew in DaluxApiService (deprecated versions)
- Shared code consistency across StreamBIM projects
- Revit extension file operations (injection)
- Tekla extension file operations (injection)
- Navisworks extension file operations (logging and error handling)
- StreamBIM diagnostics logging (sensitive data in output)
- Error handling pattern inconsistency (swallowed exceptions, info leakage)
- FluentFTP security (TLS enforcement)

Findings register

One row per finding. Ids are stable and never reused. Status is one of Open, Fixed, Accepted risk, False positive. Reviewers change status by editing this table (and the finding page); the agent respects it and only re-verifies.

Id	Severity	Title	Status	Location	First seen	Last verified	Page
SEC-001	Low	StreamBIM credential storage and FTP transfer security	Open	src/Assistant/dotnet/StreamBim/	20260925-214417	20260925-214417	SEC-001-streambim-credential-storage.md
SEC-002	High	Dalux API key handling - broken credential lookup in upload and minor HttpClient gaps	Open	src/Assistant/dotnet/DaluxCloudUpload/, src/Assistant/dotnet/DaluxCloudDownload/	20260925-214417	20260925-214417	SEC-002-dalux-api-key-handling.md
SEC-003	Info	Git history audit - no secrets found	Open	src/ across all branches	20260925-214417	20260925-214417	SEC-003-git-history-audit-clean.md
SEC-004	Critical	LISPRunner arbitrary AutoCAD Lisp script execution	Open	src/AutoCAD/dotnet/LISPRunner/LISPRunnerCommand.cs	20260926-015629	20260926-015629	SEC-004-lisprunner-arbitrary-execution.md
SEC-005	Critical	RunCommand arbitrary AutoCAD command injection	Open	src/AutoCAD/dotnet/RunCommand/RunCommandCommand.cs	20260926-015629	20260926-015629	SEC-005-rancmd-arbitrary-execution.md
SEC-006	Medium	DaluxCloudDownload path traversal via server-supplied relative path	Open	src/Assistant/dotnet/DaluxCloudDownload/DaluxCloudDownloadCommand.cs	20260926-015629	20260926-015629	SEC-006-dalux-path-traversal.md

SEC-001: StreamBIM credential storage and FTP transfer security

Severity	Low - Minor defense-in-depth gaps in FTP certificate validation and password-in-memory handling
Status	Open
Category	Secrets / Config
Location	src/Assistant/dotnet/StreamBim/Services/StreamBimCredentialProvider.cs lines 1-51, src/Assistant/dotnet/StreamBim/Services/StreamBimFtpClientFactory.cs lines 1-42, src/Assistant/dotnet/StreamBim/Models/UserCredentials.cs line 3
First seen	run 20260925-214417 at commit 4658b6b
Last verified	run 20260925-214417
Introduced	commit 183a206, 2026-09-15 (initial commit of StreamBIM uploader/downloader)
Page	Ai.Security/Findings/SEC-001-streambim-credential-storage.md

What

The StreamBIM uploader and downloader components (`StreamBIMUploader`, `StreamBIMDownloader`) use Windows Credential Manager to store and retrieve FTP credentials. This is the recommended approach for Windows applications. The FTP connection uses TLS 1.2 with Explicit encryption mode. The `StreamBimCredentialProvider` reads credentials from the Windows Credential Manager using `CredentialManager.ReadCredential()` and wraps them in a plain `UserCredentials` record. No credentials appear in logs or command-line arguments. However, there are two minor defense-in-depth observations: (1) the FTP `AsyncFtpClient` does not set an explicit `CertificateValidationCallback`, relying on .NET's default trust-store validation; and (2) password values are held as plaintext strings in memory in the `UserCredentials` record for the duration of the FTP operation, which is inherent to any application that must use credentials but is noted for completeness.

Evidence

StreamBimCredentialProvider.cs - credentials read from Windows Credential Manager, not hardcoded:

```
// src/Assistant/dotnet/StreamBim/Services/StreamBimCredentialProvider.cs line 41-51
private static UserCredentials? ReadUserCredentials(string applicationName)
{
    var credentials = CredentialManager.ReadCredential(applicationName);
    if (credentials is null ||
        string.IsNullOrEmpty(credentials.UserName) ||
        string.IsNullOrEmpty(credentials.Password))
    {
        return null;
    }
    return new UserCredentials(credentials.UserName, credentials.Password);
}
```

StreamBimFtpClientFactory.cs - FTP client with TLS 1.2 Explicit encryption, no explicit certificate validation callback:

```
// src/Assistant/dotnet/StreamBim/Services/StreamBimFtpClientFactory.cs line 14-21
var profile = new FtpProfile
{
    Host = "ftp.streambim.com",
    Encoding = Encoding.UTF8,
    Encryption = FtpEncryptionMode.Explicit,
    Protocols = System.Security.Authentication.SslProtocols.Tls12,
    Credentials = new NetworkCredential(credentials.UserName, credentials.Password),
};
```

No `CertificateValidationCallback` is set on the client. `FluentFTP` defaults to .NET's certificate validation, which trusts the system CA store.

UserCredentials.cs - plaintext password in memory:

```
// src/Assistant/dotnet/StreamBim/Models/UserCredentials.cs line 3
internal sealed record UserCredentials(string UserName, string Password);
```

Args with PasswordField attribute (prevents UI display of credentials):

```
// src/Assistant/dotnet/StreamBIMUploader/StreamBIMUploaderArgs.cs lines 5-8
[PasswordField(
    Label = "StreamBIM Credentials",
    ToolTip = "Enter your StreamBIM credentials.")]
[Required(ErrorMessage = "StreamBIM Credentials are required.")]
public UserCredentials? Credentials { get; init; }
```

No credential leakage in diagnostics - verified by grepping all `diagnostics.Log()` calls across `StreamBimFileTransferService.cs` and `StreamBimUploadDiagnostics.cs`: no password, credential, or username values are logged.

Git history - `git log -p -S "password|secret|apiKey|Bearer" -- src/Assistant/dotnet/StreamBim/` returned no results, confirming no hardcoded secrets in the commit history.

Impact

Low impact: The current implementation is security-sound. The credential storage in Windows Credential Manager (DPAPI-protected) is a best practice. The FTP connection uses TLS 1.2 Explicit encryption, which provides transport security. The absence of an explicit `CertificateValidationCallback` means the connection trusts any certificate valid in the system's CA store, which is the default and acceptable behavior for most deployments. It would only be a risk if an attacker could install a rogue CA certificate on the user's machine, which is already a high-bar privilege escalation. The plaintext-in-memory password is inherent to any application that must use credentials to perform operations; there is no practical way to avoid this. The overall implementation is a good example of credential handling in a Windows application, with the noted items being observations for defense-in-depth rather than exploitable weaknesses.

What a fix involves

No fix is required. If the organization has a strict defense-in-depth policy, the following low-effort enhancements could be considered:

- Certificate validation callback:** Set `client.Config.CertificateValidationCallback` to reject certificates that do not match expected characteristics (e.g., a specific issuer or SAN). This prevents MITM even if a rogue CA is installed. Note: this is only useful if the organization controls the CA or uses certificate pinning.
- SecureString for password:** Change `UserCredentials.Password` from `string` to `SecureString` to prevent the password from lingering in garbage-collected memory. This adds complexity as all FluentFTP APIs expect a plain string, so a conversion layer would be needed. Evaluate against the threat model before investing effort.
- Document the security model:** Add a comment to `UserCredentials.cs` and `StreamBimFtpClientFactory.cs` explaining the security choices (Credential Manager, TLS 1.2, no certificate pinning) so future developers understand the rationale.

References

- FluentFTP documentation: `FtpEncryptionMode.Explicit`, `SslProtocols.Tls12`, `CertificateValidationCallback`
- Windows Credential Manager: DPAPI-based credential storage via `System.Security.Cryptography.CredentialManager`
- OWASP Secure Storage of Sensitive Data: https://cheatsheetseries.owasp.org/cheatsheets/Secure_Storage_Cheat_Sheet.html

SEC-002: Dalux API key handling - broken credential lookup in upload and minor HttpClient gaps

Severity	High - DaluxCloudUpload bypasses credential lookup entirely, sending "Dalux API Key" as the API key; also, HttpClient lacks timeout and certificate validation callback
Status	Open
Category	Secrets / Auth
Location	src/Assistant/dotnet/DaluxCloudUpload/DaluxCloudUploadCommand.cs line 30, src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs lines 20-24 and 118-143, src/Assistant/dotnet/DaluxCloudDownload/Services/DaluxApiService.cs lines 14-24 and 207-222
First seen	20260925-214417 at commit 4658b6b
Last verified	20260925-214417
Introduced	commit 4658b6b (initial commit of upload/download extensions)
Page	Ai.Security/Findings/SEC-002-dalux-api-key-handling.md

What

The Dalux Cloud Upload extension has a broken credential lookup: it reads the API key from Windows Credential Manager but then passes the unlooked-up argument value directly to the API service. In `DaluxCloudUploadCommand.cs`, the variable `apiKey` (line 13) holds the retrieved credential, is validated for null (line 14-17), and yet `DaluxApiService` is instantiated on line 30 with `args.ApiKey` instead of `apiKey`. The `args.ApiKey` property defaults to the literal string "Dalux API Key" (the credential name), so the upload always sends the credential-name string as the actual API key, causing every upload to fail with an authentication error. The Download extension (`DaluxCloudDownloadCommand.cs` line 35) correctly passes the looked-up `apiKey` variable to its `DaluxApiService`.

Additionally, both `DaluxApiService` implementations create a raw new `HttpClient()` with no timeout, no certificate validation callback, and no explicit handling of server certificate errors. The upload service's error handler (`HandleException`) includes the full base URL in error messages, which is a minor information disclosure.

Evidence

Broken credential lookup in upload command:

```
// src/Assistant/dotnet/DaluxCloudUpload/DaluxCloudUploadCommand.cs lines 13-30
var apiKey = GetApiKey(args.ApiKey);
if (apiKey is null)
{
    return Result.Text.Failed("API key not found.");
}
// ... validation continues ...
var daluxService = new DaluxApiService(args.ApiKey, args.BaseUrl);
```

Line 30 passes `args.ApiKey` (default: "Dalux API Key" — see below) instead of the looked-up `apiKey` variable. This is a copy-paste or refactoring bug.

Args default value (the credential name, not a real key):

```
// src/Assistant/dotnet/DaluxCloudUpload/DaluxCloudUploadArgs.cs lines 10-14
[PasswordField(
    Label = "Dalux API Key",
    Tooltip = "Add organization name in the 'Name' field and Dalux API Key in the 'Password' field.")]
[Required(ErrorMessage = "Dalux API Key is required.")]
public string ApiKey { get; set; } = "Dalux API Key";
```

Correct credential lookup in download command (for comparison):

```
// src/Assistant/dotnet/DaluxCloudDownload/DaluxCloudDownloadCommand.cs lines 14-35
var apiKey = GetApiKey(args.ApiKey);
if (apiKey is null)
{
    return Result.Text.Failed("API key not found.");
}
// ... validation ...
var daluxService = new DaluxApiService(apiKey, args.BaseUrl);
```

HttpClient without timeout or certificate validation (both projects):

```
// src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs lines 20-24
public DaluxApiService(string apiKey, string baseUrl = "https://node1.field.dalux.com/service/api")
{
    _apiKey = apiKey ?? throw new ArgumentNullException(nameof(apiKey));
    _baseUrl = baseUrl;
    if (!_baseUrl.EndsWith("/"))
    {
        _baseUrl += "/";
    }
    _httpClient = new HttpClient();
    ConfigureHttpClient();
}

private void ConfigureHttpClient()
{
    _httpClient.BaseAddress = new Uri(_baseUrl);
    _httpClient.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
    _httpClient.DefaultRequestHeaders.Add("X-API-Key", _apiKey);
}
```

No `_httpClient.Timeout` set (defaults to 100 seconds), no `ServerCertificateCustomValidationCallback` configured.

Error messages include base URL (minor information disclosure):

```
// src/Assistant/dotnet/DaluxCloudUpload/Services/DaluxApiService.cs lines 120-130
private DaluxApiResponse<T> HandleException<T>(Exception ex, string operation)
{
    if (ex is HttpRequestException httpEx)
    {
        return DaluxApiResponse<T>.Failed(
            $"Network error during {operation}: {httpEx.Message}\n\n" +
            $"Base URL: {_baseUrl}\n" +
            $"Please verify:\n" +
            $"- Network connectivity\n" +
            $"- Correct API endpoint (may be region-specific)\n" +
            $"- VPN/Proxy requirements\n" +
            $"- Firewall settings"
        );
    }
    // ...
}
```

Git history check: `git log -p -S "Dalux API Key\|apiKey\|X-API-Key" -- src/Assistant/dotnet/DaluxCloudUpload/ src/Assistant/dotnet/DaluxCloudDownload/` returned no results, confirming no hardcoded Dalux API keys have been committed.

No logging of API keys: Grepped all `.cs` files in both projects for logging patterns (`diagnostics.Log`, `Console.WriteLine`, etc.) — none found. API keys are not leaked via logs.

Impact

High severity: The upload command's broken credential lookup is a functional bug that makes Dalux Cloud Upload completely non-functional — every upload attempt sends the literal string "Dalux API Key" as the API key, which will be rejected by the Dalux API. An attacker who can trigger an upload would not gain unauthorized access because the wrong credential is sent. However, the presence of this bug indicates a lack of code review on credential handling.

The HttpClient gaps are secondary: no timeout means the application can hang indefinitely on unresponsive servers (DoS potential in multi-user environments); no certificate validation callback means it trusts any certificate valid in the system's CA store, which is the default and generally acceptable. The base URL in error messages is a minor information disclosure that could help an attacker identify the Dalux deployment but provides limited value alone.

What a fix involves

1. **Fix the credential lookup bug:** Change line 30 of `DaluxCloudUploadCommand.cs` from `new DaluxApiService(args.ApiKey, args.BaseUrl)` to `new DaluxApiService(apiKey, args.BaseUrl)` to use the looked-up credential.
2. **Add HttpClient timeout:** Set `_httpClient.Timeout = TimeSpan.FromSeconds(30);` (or similar) in `ConfigureHttpClient()` in both `DaluxApiService` implementations.
3. **Consider certificate validation callback:** Add `_httpClient.ServerCertificateCustomValidationCallback = HttpClientHandler.DangerousAcceptAnyServerCertificateValidator;` only if a custom CA is used and `.DangerousAcceptAnyServerCertificateValidator` is inappropriate; otherwise keep the .NET default which trusts the system CA store.
4. **Remove base URL from error messages:** Remove or obfuscate the `_baseUrl` from the `HandleException` error string in `DaluxCloudUpload/Services/DaluxApiService.cs` to reduce information disclosure.

References

- SEC-001 (StreamBIM credential storage) — similar credential manager pattern, correctly implemented there.
- Meziantou.Framework.Win32.CredentialManager — third-party library used for credential access in both projects.
- [Microsoft HttpClient best practices](#)

SEC-003: Git history audit - no secrets found

Severity	Info - No secrets found in git history; this is a negative finding documenting the audit
Status	Open
Category	Secrets
Location	src/ across all branches
First seen	run 20260925-214417 at commit 4658b6b
Last verified	run 20260925-214417
Introduced	N/A (audit finding)

What

A full audit of the git repository was performed to check whether any secrets, credentials, or sensitive values were ever committed to the source code under src/. The search used `git log --all -p -S` with multiple patterns: password, secret, key, apiKey, BEGIN.*PRIVATE KEY, connection, and Token|AuthToken|Bearer|api_token. All matches returned either no results or only legitimate code additions (credential lookup logic, Dalux download command, OpenWorksets migration README). No actual secret values, connection strings, or private keys were found in any commit.

Evidence

Commands executed:

```
git log --all -p -S "password" -- src/      # one diff of file content updates, no secrets
git log --all -p -S "secret" -- src/        # no results
git log --all -p -S "key" -- src/           # legitimate additions only (credential lookup, API key parameter)
git log --all -p -S "apiKey" -- src/        # legitimate additions only (Dalux credential management)
git log --all -p -S "BEGIN.*PRIVATE KEY" -- src/ # no results
git log --all -p -S "connection" -- src/    # one diff of a README migration, no connection strings
git log --all -p -S "Token|AuthToken|Bearer|api_token" -- src/ # no results
```

All matches were legitimate code: DaluxCloudDownloadCommand.cs initial commit added credential lookup (using Meziantou.Framework.Win32.CredentialManager), DaluxCloudUploadCommand.cs initial commit added the same pattern, and OpenWorksets/README.md was a migration from another repo with no credentials.

Impact

None detected. No secrets are present in the git history that could be extracted by anyone with repository read access. This is a baseline security assessment confirming the repository has not been contaminated with committed credentials.

What a fix involves

No action required. The repository is clean. As a preventative measure, consider adding a pre-commit hook or CI check (e.g., gitleaks, trufflehog) to prevent future accidental secret commits.

References

- SEC-001: StreamBIM credential storage (uses Windows Credential Manager, not git)
- SEC-002: Dalux API key handling (uses Windows Credential Manager lookup)

SEC-004: LISPRunner arbitrary AutoCAD Lisp script execution

Severity	Critical - Arbitrary Lisp code execution from user-supplied file paths or inline content
Status	Open
Category	Injection / RCE
Location	src/AutoCAD/dotnet/LISPRunner/LISPRunnerCommand.cs lines 20-57
First seen	run 20260926-015629 at commit 0cbd897
Last verified	run 20260926-015629
Introduced	commit 183a206, 2026-09-15 (initial commit of AutoCAD extensions)

What

The LISPRunner extension allows users to execute arbitrary AutoCAD Lisp code through two modes: (1) specifying a file path to a .lsp script file, and (2) providing inline Lisp code. Both modes pass the user-supplied content directly to AutoCAD via `SendStringToExecute()` / `SendCommand()` without any sanitization or validation. An attacker who can persuade a user to select a malicious Lisp file or enter malicious inline code can execute arbitrary commands within the AutoCAD process — including file system access, network calls, and model manipulation. The extension treats the input as trusted and does not restrict which Lisp functions may be called.

Evidence

LISPRunnerCommand.cs - script file mode (lines 20-38):

```
private static IExtensionResult RunScriptFile(Autodesk.AutoCAD.ApplicationServices.Document doc, LISPRunnerArgs args)
{
    var scriptPath = args.LispScriptPath?.Trim();
    if (string.IsNullOrEmpty(scriptPath))
    {
        return Result.Text.Failed("A Lisp script file must be selected in SelectFile mode.");
    }

    var fullScriptPath = System.IO.Path.GetFullPath(scriptPath);
    if (!System.IO.File.Exists(fullScriptPath))
    {
        return Result.Text.Failed($"The selected Lisp script was not found: {fullScriptPath}");
    }

    var lispLoadCommand = BuildLoadCommand(fullScriptPath);
    doc.SendStringToExecute(lispLoadCommand, activate: true, wrapUpInactiveDoc: false, echoCommand: false);

    return Result.Text.Succeeded($"Queued Lisp script file for execution: {fullScriptPath}");
}
```

The file path is used directly in a `(load "...")` call. No file extension validation, no content inspection.

LISPRunnerCommand.cs - inline mode (lines 40-51):

```
private static IExtensionResult RunInlineScript(Autodesk.AutoCAD.ApplicationServices.Document doc, LISPRunnerArgs args)
{
    var inlineScript = args.InlineScript?.Trim();
    if (string.IsNullOrEmpty(inlineScript))
    {
        return Result.Text.Failed("Inline Lisp script content is required in Inline mode.");
    }

    doc.SendStringToExecute(inlineScript + " ", activate: true, wrapUpInactiveDoc: false, echoCommand: false);

    return Result.Text.Succeeded("Queued inline Lisp script for execution.");
}
```

The inline script content is sent directly to AutoCAD with no sanitization.

LISPRunnerArgs.cs confirms the input sources:

```
// src/AutoCAD/dotnet/LISPRunner/LISPRunnerArgs.cs line 8
public string? LispScriptPath { get; init; }

// line 9
public string? InlineScript { get; init; }
```

Impact

An attacker who can trick a user into running this extension with a malicious Lisp file or inline script gains code execution within the AutoCAD process context. AutoCAD Lisp can: read/write arbitrary files on the user's machine, make network requests, modify the AutoCAD database, send commands to other AutoCAD APIs, and access the Windows clipboard. This is a direct remote code execution vulnerability.

What a fix involves

1. **Remove inline mode entirely** or restrict it to a safe subset of Lisp functions (e.g., math only).
2. **For file mode:** restrict allowed file extensions to `.lsp` or `.fas`, restrict file paths to a known directory whitelist, and optionally scan file content against a blocklist of dangerous Lisp functions (`vl-file-delete`, `vlax-invoke`, `vla-`, etc.).
3. **Add logging** of which scripts are executed for audit purposes.
4. **Consider a sandbox approach:** execute scripts in a restricted AutoCAD document context with limited API access.

References

- SEC-005 (RunCommandCommand) — same pattern of unsanitized command injection into AutoCAD
- AutoCAD .NET API: `SendStringToExecute`, `SendCommand`

SEC-005: RunCommand arbitrary AutoCAD command injection

Severity	Critical - Arbitrary AutoCAD command execution from user-supplied input allows remote code execution
Status	Open
Category	Injection / RCE
Location	src/AutoCAD/dotnet/RunCommand/RunCommandCommand.cs lines 81-86
First seen	run 20260926-015629 at commit 0cbd897
Last verified	run 20260926-015629
Introduced	commit 183a206, 2026-09-15 (initial commit of AutoCAD extensions)

What

The RunCommand extension allows users to supply arbitrary AutoCAD command strings that are then executed via `SendCommand()`. The extension splits the input on newlines and executes each line as a separate AutoCAD command without any sanitization, validation, or whitelist. AutoCAD commands include not just drawing operations but also system commands, file operations, and .NET API access. This means a user can execute virtually any command available within the AutoCAD environment, including those that can affect the file system, network, or other applications.

Evidence

RunCommandCommand.cs - direct command injection (lines 81-86):

```
private static void RunCommand(string command)
{
    dynamic acadApp = Autodesk.AutoCAD.ApplicationServices.Application.AcadApplication;
    var thisDrawing = acadApp.ActiveDocument;
    thisDrawing.SendCommand(command);
}
```

RunCommandCommand.cs - multi-command execution (lines 29-54):

```
var commands = args.Commands?.Split('\n') ?? new string[] { };

foreach (var command in commands)
{
    if (string.IsNullOrEmpty(command))
        continue;

    try
    {
        RunCommand(command);
        // ...
    }
    catch (System.Exception e)
    {
        commandResults.Add(new CommandResult
        {
            Succeeded = false,
            CommandResults = command,
            ErrorMessage = e.Message
        });
    }
}
```

RunCommandArgs.cs shows the input source:

```
// src/AutoCAD/dotnet/RunCommand/RunCommandArgs.cs line 7
[Required(ErrorMessage = "AutoCAD Command(s) is required.")]
```

```
public string Commands { get; init; } = string.Empty;
```

No sanitization or validation of `args.Commands` occurs before execution.

Impact

An attacker who can supply commands to this extension gains full access to the AutoCAD command environment. AutoCAD commands can:

- Execute system commands via `Shell` or `_(shell)`
- Access the file system via `_insert`, `_wedge`, and other file-based commands
- Call .NET methods via `-.NETLOAD` and `_(.NET)` commands
- Make network requests through AutoCAD's .NET API
- Modify AutoCAD drawings and settings

This is a direct remote code execution vulnerability. The multi-line input capability means multiple commands can be chained in a single execution.

What a fix involves

1. **Implement a command whitelist:** Only allow a predefined set of safe AutoCAD commands (e.g., `LINE`, `CIRCLE`, `DIMLINEAR`) and reject all others.
2. **Sanitize arguments:** For each whitelisted command, validate that arguments contain only expected character classes (alphanumeric, specific punctuation).
3. **Remove multi-command support:** Single-command execution is safer than allowing command chains.
4. **Consider a safer alternative API:** Instead of `SendCommand()`, use the .NET API directly to perform specific drawing operations that the extension is designed for.
5. **Add audit logging:** Log all commands attempted and their results.

References

- SEC-004 (LISPRunner) — similar unsanitized execution vulnerability in Lisp scripts
- AutoCAD .NET API documentation
- OWASP Input Validation Cheat Sheet

SEC-006: DaluxCloudDownload path traversal via server-supplied relative path

Severity	Medium - Server-supplied relative path used in file output without path validation creates a supply-chain attack vector
Status	Open
Category	Injection / Path Traversal
Location	src/Assistant/dotnet/DaluxCloudDownload/DaluxCloudDownloadCommand.cs lines 368-378
First seen	run 20260926-015629 at commit 0cbd897
Last verified	run 20260926-015629
Introduced	commit 0cbd897, 2026-09-22 (DaluxCloudDownload initial code)

What

When downloading folder contents from the Dalux API, the extension uses `file.RelativePath` from the Dalux server response directly to construct local output paths (line 368-369). While `file.FileName` is used directly in single-file downloads (line 314), the relative path is a server-provided value that could contain path traversal sequences (`../`) if the Dalux API server is compromised, misconfigured, or affected by a bug. The code does not validate or sanitize the relative path before combining it with the user-specified output folder. An attacker who controls the Dalux server could write files to arbitrary locations on the user's machine within the output folder hierarchy.

Evidence

DaluxCloudDownloadCommand.cs - folder download using server-relative path (lines 367-378):

```
var relativePath = string.IsNullOrEmpty(file.RelativePath) ? file.FileName : $"{file.RelativePath}/{file.FileName}";
var outputPath = Path.Combine(outputFolder, relativePath);
var outputDir = Path.GetDirectoryName(outputPath);

if (!string.IsNullOrEmpty(outputDir) && !Directory.Exists(outputDir))
{
    Directory.CreateDirectory(outputDir);
}

await File.WriteAllBytesAsync(outputPath, content.Data, [value redacted]);
```

DaluxCloudDownloadCommand.cs - single file download using server-supplied filename (lines 314-316):

```
var outputPath = Path.Combine(outputFolder, findFile.Data.FileName);
await File.WriteAllBytesAsync(outputPath, content.Data, [value redacted]);
return DaluxApiResponse<string>.Success(outputPath);
```

The `file` object is `DaluxFile` returned from the Dalux API (`DaluxCloudDownload.Models`). The `RelativePath` and `FileName` fields are not validated against path traversal.

Impact

If the Dalux API server is compromised, an attacker could:

- Inject `../` sequences into `RelativePath` or `FileName` responses
- Escape the intended output folder by navigating up the directory tree
- Overwrite arbitrary files on the user's machine (depending on what `outputFolder` allows)
- Place malicious files in system directories if the user specifies a privileged output folder

This is Medium severity because: (1) the path values come from the Dalux server, not directly from the user, requiring a server compromise or a Dalux bug; (2) the paths are combined with the user-specified output folder, so successful traversal is limited to within that parent directory; (3) defense-in-depth is needed since the output folder is a reasonable assumption for where downloaded files go.

What a fix involves

1. **Validate relative paths:** Before using `file.RelativePath` or `file.FileName`, normalize the full path and verify it remains within the output folder.
2. **Example approach:** Resolve `Path.GetFullPath(Path.Combine(outputFolder, relativePath))` and check that it starts with `Path.GetFullPath(outputFolder) + Path.DirectorySeparatorChar`.
3. **Sanitize filenames:** Remove or reject filenames containing `..`, null bytes, or paths with `:` on Windows.
4. **Add unit tests** for path traversal attempts in both single-file and folder-download code paths.
5. The prior fix in StreamBIM (commit c5864f9) addressed path issues in that project but did not apply to DaluxCloudDownload.

References

- CWE-22: Path Traversal
- CVE history of NuGet/nupkg path traversal in Sync-ExtensionDocs.ps1 line 52-53 shows the team has this awareness for .nupkg extraction but did not apply it to Dalux downloads

Dependencies

Written by the runner from the generated bills of materials, because the agent did not finish this page. Versions need a check against advisories; this page makes no CVE claims.

Repo	Components	Bill of materials
tools-extensions-public	10	tools-extensions-public.cdx.json

Software bill of materials

One CycloneDX file per source repo, generated by the agent runner before every security review from the manifests in the checkout (OWASP cdxgen, spec 1.6). See [SBOM/README.md](#) for the index and how to regenerate. [Dependencies.md](#) is written from these files.

Source-only: versions as declared in the manifests, no license lookup, no vulnerability data. Serial numbers and timestamps are stripped so a file changes only when a dependency changes; git history is the change log. The same generator command is meant to run in the build pipeline so a released installer ships the SBOM it was built from.

Software bill of materials

One CycloneDX 1.6 file per source repo, generated by the runner (`services/aec.agents/sbom/run-sbom.sh`, OWASP cdxgen 13.0.1) from the manifests in the checkout before each security review. Source-only: versions as declared, no license lookup, no vulnerability data. The files are the authority for `Dependencies.md`; the agent reads them and never edits them. Serial numbers and timestamps are stripped so a file only changes when a dependency changes.

Repo	File	Components	Ecosystems
tools-extensions-public	tools-extensions-public.cdx.json	10	github 3, nuget 7

Regenerate by hand with `services/aec.agents/sbom/run-sbom.sh <workspace> <output-dir>`, which pins the exact cdxgen image by digest. The same command belongs in the build pipeline so a release ships the SBOM it was built from.